

2.System Security

#System Security

System Security : 사고나 악의적인 액세스, 사용, 수정, 파괴, 노출로부터 컴퓨터 하드웨어와 소프트웨어를 보호
Confidentiality(기밀성), Intergrity(무결성), Availability(가용성) by NIST in 1995

System :

Hardware : 모니터, 하드디스크, 메모리, router switch 등

Software : OS, DB, web service 등

Firmware : BIOS, embadded system 등

ROM : Read Only Memory

Non-volatile

RAM : Random Access Memory

Volatile

Account managemnt(계정 관리) : to Identify user with appropriate privileges

Session management(세션 관리) : to handle multiple requests to web-based application/service from a single user

Access control(액세스 관리) : to make sure user have appropriate access to sensitive/important data

Permission management(권한 관리) : to allow users to grant or revoke permission on the system resources

Log management(로그 관리) : to handle(gather, sort, store) log data from variety of system sources

Vulnerability managemnet(취약점 관리) : to recognize, classify, and alleviate vulerabilities in software and firmware.

Identification(식별) : 사람이나 사물의 identity를 identify하는 행동

Sth a person KNOWS : 비밀번호, PIN, 엄마 이름, 아빠 이름

Sth a person HAS : key, swipe card, access card, OTP

Sth a person IS : biometric

SOMEWHERE a person s : 특정 IP, callback; 어디로부터 접근했는가?

Authentication(인증) : 컴퓨터 시스템 유저의 identity를 인증하는 행위

비밀번호 인증

2-Factor Authentication

Multi-factor Authentication (Biometric)

Authorization(인가) : 특정 리소스에 접근할 수 있게 하는 기능

#Account Management(OS)

user account : 설정의 모음이다.파일과 폴더 control. 허용된 작업을 함. 장치와 리소스 사용

user group : 여러 user account의 모음. 같은 권한을 가지고 있도록 설정을 공유함

Windows OS에서 중요한 account groups

Administrators : 모든 권한 가지고 있음

Power Users : 대부분의 권한가지고 있음. 계정 생성/삭제 가능. but Admin 그룹의 설정은 못바꿈

Backup Users : 백업과 복구를 할 수 있다. 파일 권한과 상관없이 가능

Users : 일반적인 작업 가능. 계정 생성 불가, 프로그램 설치 불가.

Guests : 임시 프로필. 로그아웃하면 삭제됨

Administrator Local Security Policy

- Administrator 이름 바꾸기.
- 잘 알려진 이름이면 바꾸기
- 필요 없는 계정들 지우기

윈도우 OS 계정관리

net user <user_name> // 유저에 접근

net user UserLoginName /add /active:yes //UserLoginName이란 계정 추가

net localgroup administrators Visitor /add //Visitor를 administrators 그룹에 추가

Powershell : CMD의 개선판. shell script 사용 가능

OOP : CMD보다 복잡한 task을 할 수 있는 간단한 구현.

\$i=0; Get-Content Large_logs.txt -ReadCount 100 | %{\$i++;\$_} | Out-File output-%i.log; Write-Host \$i

Linux나 Unix-like OS의 계정 관리

Linux 디렉토리의 뒤에 /가 없으면 파일, 있으면 디렉토리

'root' : 시스템 관리자 계정

/etc/passwd에 계정 정보 들어있음

tony : x : 1003 : 1003 : Tony Smith ... : /home/tony : /bin/bash

유저이름 : 비번 : UID : GID : 주석 : 홈디렉토리 : 프로그램이나 셸

유저이름 : 1~32 char

비밀번호 : /etc/shadow에 해쉬 암호화되어 저장. x는 비밀번호가 설정되어있음을 의미

UserID : 유저 ID. 각자 다른 ID 부여. 일반적으로 1000부터 부여. 999까지는 예약되어 있음.

GroupID : 그룹ID /etc/group에 저장

user information(GECOS) : 주석

home directory : 홈 디렉토리를 보여줌. user가 할당받은 페이지

프로그램이나 셸 : 사용할 셸 설정(default : bash)

/etc/group에 그룹 정보 들어있음

wheel : x(*) : 10 : user [, user2, user3, ...]

그룹이름 : 그룹비번 : GID : 유저 리스트

/etc/shadow에 비밀번호 정보 들어있음

mail : \$6\$4~~~ : 99999 : (4) : (5) : (6) : (7) : (8)

유저이름 : 암호화된 비번 : 마지막 바꾼날 : 최소기간 : 최대기간 : 경고 : 비활성화 : 파기

#UNIX Architecture

Kernel & Shell : OS의 중요한 구성요소

Kernel : process, memory, I/O 관리를 제공하는 interface. OS에서 중요한 컴포넌트들이 올라가 있는 공간

System call : 커널이 제공하는 서비스에 접근하기 위한 interface. 시스템에서 필요한 기능을 함수로 불러옴

Shell : Command line input을 해석하고 필요한 프로그램을 호출시키는 interface. calls

BS Shell 등등 다양한 종류가 있음.

하드웨어를 보호하기 위해 호출하는 방식으로 구성.

32bit x86 아키텍처라고 가정해보자.

메모리 주소 표시할 때 최대 32까지 표시할 수 있음. 따라서 표시할 수 있는 크기 $2^{32} \sim 4 \times 10^9$ == 약 40억개.

처음 3GB는 0x00000000부터 0xBFFFFFFF까지 유저가 사용 가능.

마지막 1GB는 0xC0000000부터 0xFFFFFFFF까지 OS커널이 예약.

Bit연산자 : << = 곱하기 2 >> = 나누기 2

Stack : First In Last Out (선입후출); e.g. Ctrl+Z

Queue : First In First Out (선입선출)

#Memory Layout + Allocation(UNIX)

Physical address : 물리적인 주소

Virtual address : 실제 물리적인 위치에는 떨어져 있지만, 마치 연속되어있는 것처럼 쓸 수 있는 논리적인 주소

Paging : 메모리를 관리하기 위해서 동일한 크기의 블록으로 저장 공간을 나누는 것. 논리 페이지와 물리 페이지로 맵핑함. 그 사이는 페이지 테이블에 의해 유지. 논리 주소를 물리 주소로 변환하는데 사용됨.

Virtual memory:

C에서의 메모리 관리

malloc : heap에서 메모리를 할당

free : heap에서 메모리를 해제

C소스코드

C 구문은 함수들로 구성됨

파일들의 집합으로 저장됨 (.c .h 등)

실행 모듈(Executable module)

컴파일러에 의해 2진수로 변환되고 파일로 저장됨.

.obj 파일을 생성해냄. 라이브러리 파일들의 함수들을 obj 파일로 연결해줌.(연결해 주는 것 linker)

>> compiler와 linker가 합쳐져 exe 파일을 만들.

TDBH S

Text : 실행 가능한 코드와, 상수

Data : 초기화된 **global&static** 변수 (선언과 동시에 initialized된 경우에만 해당)

BSS : 초기화되지 않은 **global&static** 변수

Heap : dynamic(동적) 메모리 Heap-Dynamic

Stack : local 변수 - 함수 Scope 안에서만 살아남음! Local-Stack

Push-입력 Pop-출력 Top-포인터

global, static 변수 - 프로그램이 끝날 때까지 사용가능

local 변수 - 함수가 return 될 때까지 사용 가능

malloc된 것 - free 될 때까지

+추가

void* : void 포인터; 모든 타입의 변수를 가리킬 수 있는 포인터. 하지만 사용시에 데이터타입 명시해야 함.

size_t : sizeof()에 의해 반환되는 unsigned int 타입의 변수

void* malloc(size_t size) : size라는 변수의 메모리 크기만큼을 할당하는 함수

int * =(int *) malloc(sizeof(int));

```
char * string = "hello"; // string->data "hello"-> Text
int iSize;                // iSize - > BSS
char* f(void){
    char* p;               //p -> stack
    iSize=8;
    p=malloc(iSize);       //malloc(iSize) -> heap
    return p;              //
}
```

+포인터 정리

```
int a=1;
int * pa=&a; // pa라는 int형 포인터 변수를 선언하고 a의 주소값을 넣어라
printf('%d',pa); //pa의 주소를 출력
printf('%d',*pa); //pa가 가리키는 것을 출력(값에 접근)
```

Leak : 함수 내에서 malloc을 하고 free를 하지 않은채로 return을 하면 leak가 발생.

Dangling Pointer : 함수 내에서 지역변수의 주소를 반환하는 경우, 함수가 return되면 지역 변수는 메모리에서 사라져서 반환된 pointer는 유효한 메모리를 가리키고 있지 않음! 데이터를 가리키는 곳이 사라져버림.

3. Database(SQL)

DML : Data Manipulation Language; DB에 데이터를 받거나, 저장하거나, 수정하거나, 삭제하거나, 삽입하거나 업데이트.
SELECT INSERT UPDATE DELETE

create ~ 데이터베이스를 만들어라

source ~ sql 파일에서 불러와라

#Table : DB안에 모든 데이터를 가지고 있는 DB 객체. 엑셀처럼 행과 열로 이루어짐.

1줄의 Row = Record = Tuple
1줄의 Column = Field

#Use

>USE Mysql //Mysql 테이블을 쓰겠다.

#Desc / Describe

테이블이 어떻게 구성되었는지를 보여라

#Select

SELECT [ALL * | DISTINCT]
FROM tablename
WHERE filter conditions
GROUP BY field/column name
HAVING filter condition
ORDER BY field/column name [ASC/DESC]

#select job from emp; --emp테이블의 job을 다 select해라

#distinct

중복된 것 정리해서 한번만 세라.

select distinct job from emp; --emp테이블의 job을 select해라 단, 중복된것 정리해서

#count()

NULL이 아닌 것의 개수를 세라.

select count(*) from emp; --emp 안에 있는것의 개수를 세라!

select count(comm) from emp; --emp 안에 comm에 NULL이 아닌 것의 개수를 세라!

#Where 조건

#등호

select * from emp

where deptno = 10; -- emp중에 deptno가 10인 것만 select.

```
select * from emp
where sal >= 2500; --emp중에 sal이 2500과 같거나 큰 것만 select.
```

```
select * from emp
where ename='KING'; --emp중에 ename이 'KING'인 것 select.
```

#%, _ 검색

```
select empno, ename from emp
where ename like 'S%'; --emp 중에 S로 시작하는 row의 empno, ename col을 select.
```

```
select empno, ename from emp
where ename like '%T%'; --emp 중에 T가 들어있는 row의 empno, ename col을 select.
```

```
select empno, ename, job from emp
where job like '_____man'; --emp 중에 job앞에 5자리 무언가가 있는 row의 empno, ename, job
을 select.
```

#or, and, in, between

```
select empno, ename, comm from emp
where comm=300 or comm=500 or comm=1400; --com이 300이거나 500이거나 1400이상인 것.
where comm IN(300,500,1400); --com이 300이거나 500이거나 1400인 것.
where sal between 1200 and 3500; --sal이 1200에서 3500인 것
```

```
select ename, empno, job, deptno from emp
where deptno = 30 AND job='manager'; --and
```

```
select ename, empno, deptno from emp
where NOT deptno=30; --deptno 30
```

```
select ename, empno, deptno from emp
where comm NOT in(300, 500, 1400);--NOT(300, 500, 1400)
```

```
select empno, ename, sal from emp
where sal NOT between 1200 AND 3700; --NOT(1200 and 3700)
```

```
where sal <=1200 or sal>=3700; -- 1200과 37000 사이
```

```
where sal NOT BETWEEN 801 AND 4999; -- 801보다 작고, 4999보다 큼
```

```
where sal <= 800 or sal >= 5000; --800보다 작거나 같고, 5000보다 크거나 같음
```

#null인가?

```
select ename, job from emp
where mgr IS NULL; // mer에 값이 없는 것을 골라라!
where mgr IS NOT NULL; -- mer에 값이 없는 것을 골라라!
```

#GROUP BY

```
select deptno, avg(sal) from emp
deptno;      --deptno를 기준으로 그룹지어라! 단 avg, sum, min, max 같은 것들을 사용해야 함.
              --deptno 값을 기준으로 deptno값이 같은것들끼리 연산!
```

#HAVING; Group by가 있어야만 나올 수 있음!

```
select deptno, avg(sal) from emp
group by deptno
having avg(sal)>=2000; --group by 한 것 중에서 avg(sal)한 결과가>=2000인 행만 sorting해라! where
사용하면 안됨!!
```

```
select deptno, avg(sal) from emp
where sal>=1000
group by deptno
having avg(sal)>=2000;      --sal이 1000이 넘는 사람들을 deptno로 그룹화 해라. 그리고 그중
avg(sal)이 2000이 넘는것만 출력!
```

#ORDER BY

```
order by [colname]; //오름차순으로 정렬해라!      작은거>큰거
order by [colname] asc; //오름차순으로 정렬해라!   작은거>큰거
order by [colname] desc; //내림차순으로 정렬해라!   큰거>작은거
```

```
select empno, ename, sal from emp
order by sal desc, ename asc; //sal을 내림차순으로, ename을 오름차순으로
```

#limit ; 상위 n개만 출력!

```
select empno, ename, sal from emp
order by sal desc, ename asc limit 5; //sal을 내림차순으로, ename을 오름차순으로
```

#JOIN

두 개 테이블에 있는 정보를 불러오기 위해서는 그냥 불러왔을 때 불필요한 정보도 같이 출력됨

Primary Key : NULL 값이 있으면 안 되고, UNIQUE해야 함.

Foreign Key : 다른 Primary key를 참조 두 테이블간 관계를 정의.

#INNER JOIN : A 테이블과 B테이블에서 같은 것(중복되는 것) 만 가져옴.

한계 : 양쪽 테이블에 존재하는 것들만 추출 가능

```
select ename, dname from emp, dept
where emp.deptno = dept.deptno;
```

```
select e.ename, d.dname from emp e, dept d // 각각의 테이블에 Alias를 사용함. emp==e, dept==d
where e.deptno = d.deptno;
```

```
select e. ename, d.dname from emp e
INNER JOIN dept d
ON e.deptno = d.deptno;
```

```
mysql> select e.ename, m.ename "manager" from emp e, emp m //ename을 출력하되 "column 이
름을 manager로 출력
where e.mgr=m.empno
order by m.ename;
>> NULL 값이 포함되어있는 row는 출력되지 않음!
```

#OUTER JOIN : 포함되는 쪽(Left, Right)에 있는 것을 기준으로 NULL 값이 있어도 출력됨

#Subqueries 서브쿼리

```
select dname from dept
where deptno=(select deptno from emp where ename='JONES');
```

ANY(or SOME) : 서브쿼리가 반환하는 값 중 하나라도 조건을 만족하면 참. OR

IN(NOT IN) : 서브쿼리가 반환하는 값 중 하나라도 조건을 만족하면 참.

EXISTS(NOT EXISTS) : 값이 존재하는가? (NULL 값이 아닌가?)

ALL : 모든 조건을 만족하는가? AND?

https://www.youtube.com/watch?v=JkfUvuKL2UM&ab_channel=CodeEra

subquery가 더 느리다! > join을 써야

#변수설정

```
set @val;
```

#Create

```
create table persons(
    personID int,
    Firstname varchar(255),
    LastName varchar(255),
    Address varchar(255),
    City varchar(255),
    Primary key(personID)
    Foreign Key(OrdereID) Reference 다른테이블 이름(다른테이블 col name)
```

#INSERT

```
insert into 테이블이름(col1, col2,...)
values(val1, val2...)
```

#UPDATE


```
update 테이블이름  
set col1=val1, va2=val2  
where 조건
```

```
}
```

char(size) : 고정된 길이의 문자열. 기본은 1바이트. 최대 32바이트
varchar(size) : 가변 문자열. 최대 65536(2^{16})바이트. 마지막에 \0 널문자 있음.
text(size) : 최대 65535바이트. LONGTEXT는 43억개
tinyint(size) : 1바이트 정수. -128~127. unsigned는 0~255
bool : 0는 false, 0 이외는 true
int(size) : 중간 정도 정수. 4바이트~~ 2^{16} 비트~~43억
bigint(size) : 큰 정수
float(size, d) : size에는 n자리, d에는 소수점을 어디에 찍을지 정하는 것.
ex) float(6,2) : 234.56을 저장. 소수점도 자리 차지!
date(~,~) : 1000년 1월1일~9999년 12월31까지

#view

```
create view 뷰이름 as  
select 컬럼, 컬럼  
from 테이블이름  
where 조건
```

4. Database (5주차 1:45:00~)

#데이터베이스 계정 관리

OS Authentication : OS인증을 데이터베이스 관리자로할 수 있다.

SYSDBA 권한: 데이터베이스 접속 권한

Oracle에서 dba(Linux), ORA_DBA(Windwos)가 계정임.

유명한 DB : Oracle, MySQL, MS SQL, PostgreSQL, MongoDB, MariaDB, HBase, IBM DB2,

미리정의된 관리자 계정 :

Oracle : SYS&System

MSSQL : sa

MySQL, MariaDB: root

IBM DB2 :db2inst1(UNIX), db2admin(Win)

#세션 관리

HTTP - Connectionless/stateless; 연결시키고 응답이 오는 순간 끊어버림.

요청할 때만 통신TCP/IP 바탕으로 이미지, html파일 등 전송.

Cookie - Key-Value로 이루어진 작은 텍스트파일. 컴퓨터에 저장됨

session cookie : 브라우저를 닫으면 사라짐. 1회용 쿠키

permanent cookie : 더 긴 기간 저장. 사용자 식별. 이전 방문 상태를 추적(tracking cookie)

Session - 사용자가 웹 사이트에 로그인하고 탐색하는 동안 유지. 서버에 저장되는 것.

보통은 2~30분정도 행동이 없으면 세션을 끊어버림

#세션에 대한 보안 이슈

Session Fixation : 인증된 유저의 세션을 하이재킹함.

Cross-site scripting(XSS) : 악의적인 스크립트를 주입.

Cross-site request forgery(CSRF) : 웹 보안 취약점. 의도하지 않은 행동을 하게 함.

#쿠키와 세션의 차이

쿠키 : 4kb	클라이언트에	자동으로 파기시점 설정	빠름	덜 안전
세션 : 최대 5mb	서버에	파기 시점 설정됨. 끄면 파기.	느림	안전

#엑세스 컨트롤

파일에 접근할 때 거부해야 함 (confidentiality; 기밀성)

파일을 보호해야 함 (intergrity; 무결성)

주체 : 유저, 프로세스...

객체 : 파일, 리소스, 테이블...

엑세스 권한 : 읽기 쓰기 지우기 실행하기 업데이트하기

#보안 정책

Discretionary Access Control(DAC); 임의적 접근 방법. 가장 많이 사용. 임의로 세팅 가능

Mandatory Access Control(MAC); 의무적 접근 방법. 정보, 군에서 사용

Bell-Lapadula(기밀성 중요 모델; no read up, no write down)

Biba(무결성 유지; no read down, no write up)

Role-Based Access Control(RBAC) : 역할기반 접근 방법. 복잡한 조합 필요 시 사용

	DAC	MAC	RBAC
Grant access rights/privilege	Data owner (Administrator)	System	Central Authority
Access determination	Rank/Position	Security Label	Role
Policy	Flexible	Rigid	Flexible
Advantage	- Easy to implement -	Higher security (Only admin can access/alter all controls)	Simple management of the security policy
Disadvantage	- Vulnerable to Malware - Hard to scale up with large number of subject/objects	Hard to implement	Labor-intensive setup
Use case	ACL	Firewall	HIPAA

#RBAC

RBAC0 : 기본 모델

RBAC1 : 역할 계층. 상속 가능

RBAC2 : 제한 생김. 상호배타역할. 선수요소

RBAC3 : RBAC1+RBAC2

현재는 RBAC3 가장 많이 사용.

DBMS에서의 RBAC : GRANT로 권한 부여

#DAC

Access control Matrix 사용.

각 대상별로 어떤 권한을 가지는지 2차원 테이블로 저장. > 사람들이 많아지면 굉장히 커짐.

대안

- Capabilities(C-list) : Subject를 기준으로 권한 저장 각 행을 분류함
- Access Control list(ACL) : Object를 기준으로 권한 저장

#안전한 OS를 위한 테크닉

유저 계정 : 계정 수 최소화, 최소한의 권한만.

파일 시스템 : read-only. deny를 default로

계정 정책 : 강력한 비밀번호

네트워크 서비스 : 필요한 서비스만. 최소화.

OS 최소화 : 필요없는 프로그램 삭제.

시스템 패치 : 최신, 필수 패치

&6주차 20:00~

#윈도우에서의 Permission 관리

6가지 Permission(권한)

Full control : 읽기 쓰기 수정 실행 특성변환을 허용함. ownership 가짐

Modify : 읽기 쓰기, 삭제 허용

Read&Execute : 보기,리스팅하기, 실행 ONLY

Read : 보기,리스팅하기

Write : 쓰기, 추가하기

Special Permission : Volume Shadow, NTFS(최대 128기가인 파일 시스템) quotas

Volume Shadow : 서비스와 시스템 백업, 복구를 위한 copies 지점 생성

NTFS quotas : 유저가 스토리지 용량을 너무 많이 쓰지 않도록 쿼터를 지정

권한 충돌 시 규칙

1. Deny가 allow보다 앞선다. (**explicit deny**; 명시적 거부)
2. Direct하게 적용된 permission은 상속받은 권한을 앞선다, (**explicit allow**; 명시적 허용)
다만, full control일 경우는 무시. full control 우선
3. 가까이 있는(near relative) 속성을 상속받는다.
4. 같은 레벨에 다른 유저그룹에 속해있으면 속성을 누적할 수 있음

Explicit Deny > Explicit Allow > Inherited Deny > Inherited Allow

Explicit permission : 객체가 생성될 때. 또는 유저가 생성할 때

Implicit permission : 부모 객체의 자식으로 받은 권한들

#유닉스/리눅스에서 Permission 관리

0/	000/	000/	000
타입	유저	그룹	외부

ls : 디렉토리 리스트를 보여줘

파일 타입

d : 디렉토리 p : named pipe(통신) - : 파일 l:링크 s:소켓 c:캐릭터 디바이스

rwX : readable/writeable/executable

8진수 값으로 표현함. **r=4 r=2 x=1**

UNIX에서의 기본 permission

unmask(022) : -/x/x

file(666>644) : rw/rw/rw > rw/r/r

directory(777->755) : rwx/rwx/rwx > rwx/rx/rx

execute한다 : 디렉토리에 들어갈 수 있다.

r/w 한다 : 파일(문서)를 읽고 쓴다.

UNIX에서 권한 바꾸기 : chmod 명령어 사용

유저 권한 추가 : `chmod u+rw <파일이름>`

그룹/other 권한 추가 : `chmod g+r <파일이름>` `chmod o+r <파일이름>`

권한 삭제 : `chmod g-rw <파일이름>`

권한 변경 : `chmod 644 <파일이름>` `chmod 777 <파일이름>`

Link란? - 간단히 말하면 바로가기

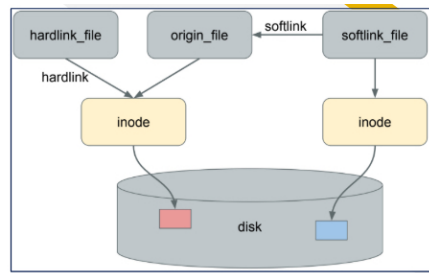
ln 명령어 사용

hard link(하드링크) : `ln 파일1 파일2`

원본파일과 같은 inode를 가리킴. “복사”

soft link(소프트링크) : `ln -s 파일1 파일2`

새로운 inode를 가리킴 “포인터”



Parameter	Soft Link	Hard Link
Inode number	Different <i>inode</i> number than the original file	Same <i>inode</i> number as the original file
Directory	Soft links can link directories	Hard links can not link directories across
Original File deletes	The link will not operate if the original file is deleted since it does not access the data in the original file. It's nothing more than a shortcut to the original file	The Hard link will continue to operate even if the original file is deleted since it accesses the same data as the original
Speed	Slower	Faster than soft links
Memory Consumption	More	Less

Inode 구조

:객체를 묘사하는 데이터 구조체. 파일과 디렉토리 포함. 메타데이터 포함.

메타데이터 : 파일과 디렉토리 위치 저장. 이름, 소유자 계정, 권한

direct하게 접근하기 보다는 포인터를 통해서 데이터 블록들을 관리함.

double Indirect, triple indirect를 사용해서 주소를 관리하면 더 큰 용량을 관리할 수 있음.

#DBMS에서의 권한

유저 생성

`create user '유저이름'@'%' identified by '비밀번호';` // %는 외부IP. 로컬이면 localhost라고 적기

허가(Grant)

`grant all privileges on *.* to '유저이름'@'서버';` (*.*: 모든DB, DB이름 : 특정DB에만)
(insert, select, update...)

적용

`flush privileges;`

권한/유저 삭제

```
revoke all privileges on *.* from '유저이름'@'서버';      //권한 삭제
drop user '유저이름'@'서버';                          //계정 삭제
```

#DB 보안 취약점

KISA에서 취약점에 대한 보고서를 발간함.

클라우드 취약점 체크 가이드, MySQL 시스템 취약점 체크 가이드

#로그 관리

AAA : 네트워크와 관리적 액세스 관련된 Cisco의 강력한 보안 권고

Authentication(인증) : 유저를 식별하는 방법 제공(로그인)

Authorization(권한) : 유저에 대한 엄격한 권한 조치

Accounting(감사) : authentication과 authorization을 계속해서 저장하고 track.

System monitor logs : MS에서 제공하는 로그 모니터 툴. 따로 설치 해야함.

다양한 활동에 대한 로그를 생성.

악의적 행동이 로그에 의해서 식별될 수 있음.

Windows security logs : 보안에 특화된 로그(로그인, privilege 변경 등)

Windows system logs : 이벤트 뷰어. 윈도우에 빌트인 되어있음.

Netflow logs : CISCO에서 만듦. 네트워크 관리 툴.

PCAP(Packet Capture) logs : 패킷 데이터를 수집. 트래픽을 볼 수 있음. IP주소/도메인...

Firewall logs : wf.msc\로 실행 가능. inbound 규칙(보통 거부), outbound 규칙을 설정 가능.

Proxy logs : 프록시 서버 기반 네트워크에 접속하는 유저와 앱의 로그.

프록시 서버: 유저와 인터넷 사이에 gateway 역할. child control, 보안, 속도, 차단된 콘텐츠 접속을 위함,
(Forward) Proxy : 클라이언트를 보호하기 위함.

Reverse Proxy : 서버를 보호하기 위함 - 캐시 등을 이용해 속도를 높이기 위함.

Browser History logs : 웹페이지와 앱 방문 기록을 추적.

DNS logs : DNS 서버를 통해 받은 DNS 데이터를 제공. DNS 공격에 대한 정보 식별(하이재킹, 터널링, DoS)

#백업(dump)

DB 백업 : mysqldump -uroot -p --alldatabases(또는 db이름) > all.sql

DB 복구 : mysqldump -uroot -p --alldatabases(또는 db이름) < all.sql

Binary log : DB에서 업데이트된 모든 쿼리를 담은 로그

%MYSQLROOT%DATA에 있는 파일을 SQL파일로 바꿔야 읽을 수 있음.

• MySQL Backup using job scheduler

- Backup at 00:01 AM on the 1st Sunday of every month

```
SCHTASKS /Create /SC MONTHLY /MO first /D SUN /ST 00:01 /TN MySQL_BackUp /TR C:\mysql-8.0.28-winx64\backup.bat
```

한달에 한번 첫 번째 일요일 00:01에 백업

backup.bat으로 저장됨.

Error Log : MySQL 시작이나 쿼리 오류에 대한 메시지를 저장

General Log : 기본적으로 OFF되어있음. 모든 쿼리 내용 기록.

#Web Server logs

Apache web server : %로 포맷스트링 custom해서 쓸 수 있음. 대소문자 구별 주의!

NGNIX :

Tomcat :

Node.js :

#취약점 관리

Vulnerability Patch : OS, SW, App, 네트워크 구성요소의 취약점을 체크하고, 취약점을 막음.

Patch Management : Lifecycle 관리의 일부

8가지 Patch Management practice

- 인벤토리 만들기(할 일 정리)
- 시스템 카테고리화
- 우선순위 높은 패치를 식별, 빨리 처리
- 패치 테스트 해보기
- 패치 타임라인 설정. 언제 할지
- 취약점을 스캔하기. 지속적으로
- 패치 자동화 소프트웨어 사용하기
- 패치 리포트 받기

windows에서 gpedit.msc에서 자동 소프트웨어 업데이트 설정 가능.

#모바일 보안

모바일 디바이스의 사용이 늘어남.

BYOD(Bring your own device) : 기업을 보호하기위한 모바일 디바이스 관리
클라우드에 의존.

모바일 OS 안드로이드, iOS 독점

2011 노키아와 블랙베리의 몰락

#Apple iOS 보안

Secure boot chain : 부트를 하기 위한 여러 가지 보안 장치.

boot ROM : 제작 당시 built in 해놓은 장치.

Low level bootloader : 펌웨어 서명이 되어있는지 verify

iBoot : ios 커널에 대한 신호를 verify

Sandbox : 접근 권한에 대한 access strategy. 사용 가능한 리소스에 대해 explicit 해야함.

#안드로이드 보안

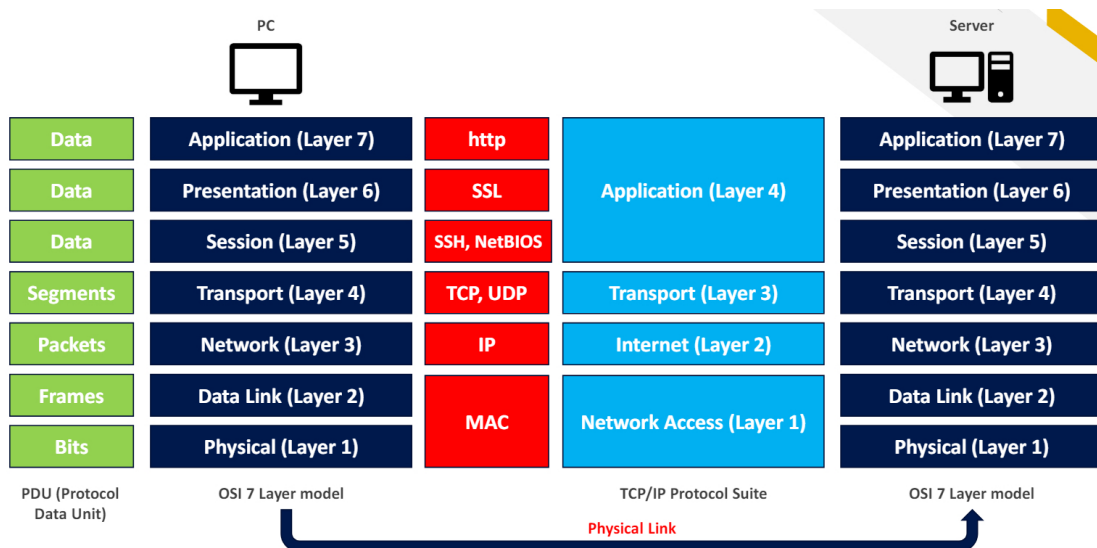
JAVA 기반. 가상머신 위에어 구동

Dalvik : 모바일에서 돌아가는 가상머신 4.4 킷캣부터 구동

3. Network (5주차 1:45:00~)

#OSI 7 Layer

Open system interconnection: communication system에서 7개의 단으로 구성된 모델



#1st. Physical Layer

제일 저층 레이어. 데이터의 가장 기본적인 형태 bit-level

RJ45선이나 무선으로 전기신호 전송.

구성 HW : 네트워크 인터페이스 카드, 허브(들어오는 신호를 뿌림. broadcast. L1-dummy, L2-switching), 리피터(신호 증폭), 케이블.

전선을 꼬거나 동으로 감싸서 저항을 줄여 유실이 되지 않고 노이즈 적게 함.

전송 매체의 종류

Simplex : 단방향. 마우스, 키보드

Half-duplex : 전송과 수신을 번갈아 가면서. 동시에는 안 됨. 워키토키.

Full-duplex : 양방향 통신.

#2nd. Data Link Layer

모든 정보의 packet이 에러없이 통과하였는지 확인.

구성 HW : Network Interface Card, Switch, Bridge

2단계의 3개 주요기능

전송 에러를 다룸

데이터 흐름을 조절

네트워크 레이어로 잘 구성된 인터페이스를 제공함

Layer2 Switch의 충족 조건

MAC 주소 테이블

packet forwarding : 패킷 전송

packet buffer : 잠시 저장. 오류 발생 시

#3rd. Network Layer

호스트들 간 데이터를 전송하는 데 도움.

데이터 패킷들을 위한 가장 빠른 경로를 찾음(패킷 라우팅)

디바이스들이 unique한 주소를 가지고 논리적으로 addressing하게 함.

기본 구성 HW : Router (각각의 네트워크 하나에 **최소 1개의 라우터**가 있어야 함)

TCP/IP 세팅이 되어있어야 함!

IP Address

Subnet Mask

기본 Gateway

IP 주소(IPv4)

32비트로 구성. 각각의 decimal 4개의 묶음으로 표현. <-> IPv6 128비트

TCP/IP에서 unique하게 식별됨.

2진수에서 8비트씩(octet)끊어서 점으로 나눔. ex) 192.168.123.132

네트워크 주소 : 192.168.123.0

호스트 주소 : 0.0.0.132

>>이 두 개를 합쳐서 IP주소가 됨.

서브넷 마스크 Subnet Mask(IPv4)

전체 네트워크를 보는 것이 아니라, 일부만 보고 결정하겠다.

어디까지가 네트워크고 어디까지가 호스트인가?

TCP/IP에서 unique하게 식별됨.

서브넷 마스크가 255.255.255.0이면 24비트-Network, 비트-host

192.168.123.0 < 0을 붙여 네트워크임을 알 수 있게 함. Network ID

192.168.123.255 <255을 붙여 Broadcast임을 알 수 있게 함.

>>실제로 254개만 쓸 수 있다.

Network Class

네트워크 크기에 따라 분류하는 방식

DHCP : Dynamic Host Configuration Protocol

클래스 서브넷마스크

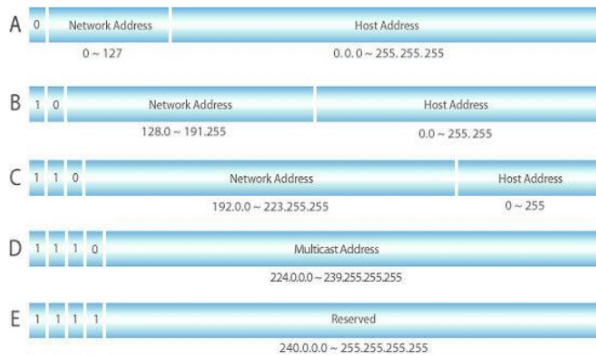
A : 255.0.0.0(기본) : 첫 옥텟 0~127(첫 자리 0제외 7비트만)

B : 255.255.0.0(기본) : 첫 옥텟 128~191(첫 자리 0 제외 6비트만)

C : 255.255.255.0(기본) : 첫 옥텟 192~223(첫 자리 0 제외 5비트만 사용)

D : 멀티캐스팅 앱에 예약

E : research용으로 예약



IPv6

IPv4의 단점을 극복하기 위해서 만들 더 많은 주소 표현 가능

128비트 사용. 8개 그룹으로 hex 숫자 사용.

2001:0DB8:AC10:FE01:0000:0000:0000:0000 == 2001:0DB8:AC10:: (생략가능)

MSB : Most Significant Bit; 앞부분 <-> LSB : Least Significant bit; 뒷부분

IPv4>IPv6로 변환 시 앞을 다 0으로 채운다!

IPv4 vs IPv6

IPv4 : 32bit	Manual, DHCP	checksum, 패딩 O	broadcast
IPv6 : 128bit	자동으로 renumber	checksum, 패딩 X	multicast, anycast

#TCP/IP와 서브넷

VPN : 가상사설망

NAT(Network Address Translation) : IP 주소를 바꿔주는 역할. IPv4의 부족한 자원을 보존하기 위해서.
보안적인 장점도 있음.

PAT(Port Address Translation) : 시스템별로 포트를 할당. 라우팅 테이블을 보고 서버로 연결해줌

CIDR

IP뒤에 슬래시를 하고 표기. Network ID를 몇비트 쓰는지 알 수 있음

/8이면 network ID 8비트 사용 /0이면 0비트 사용

Subnetting

네트워크를 더 작은 네트워크로 나누는 것

Class C address 199.1.10.0 masking 255.255.255.224 (11111111.11111111.11111111.11100000)

네 번째 octet에서 서브넷 네트워크 ID로 3비트>8개 사용. 호스트는 5비트>32개 가능.

호스트 주소는 0~31, 32~63, 64~ 등으로 나뉘짐. 그런데 첫 번째와 마지막주소는 network id와 gateway로 써야하므로 32-63을 쓴다고 하면 31-62만 쓸 수 있다.

C Address	199.4.10.0	11000111.11000000.01000001.00000000
Standard mask	255.255.255.0	11111111.11111111.11111111.00000000
Mask	255.255.255.240	11111111.11111111.11111111.11110000

y = 4 (borrowed bits)

x = 4 (bits left in host field after borrowing)

Formulas:

➤ 2^y = # of usable subnets

➤ $2^x - 2$ = # of usable hosts per subnet

➤ 2^4 = 16 (usable subnets)

➤ $2^4 - 2 = 14$ (usable hosts per subnet)

가장 오른쪽에 있는 비트가 중요한 비트임.

Gateway : 어떤 라우터를 통해서 빠져나가는가?

Unicast : one to one 일대일

Broadcast : one to all 가능한 모두에게 Ping, ICMP등...

Multicast : one to many 224.0.0.0부터 사용 D클래스에서 사용

LAN (Local Area Network) : 내부 네트워크에서 근거리 통신. 같은 IP 범위에서 사용.

ARP (Address Resolution Protocol) : MAC 주소를 매핑하는 프로토콜. 특정 IP 주소에 대한 MAC 주소를 찾기 위해 사용됨.

#4th Transport Layer

네트워크에 있는 컴퓨터 간 통신을 제공.

TCP(Transmission Control Protocol) : 두 단말이 연결을 하고 데이터를 전송. 전송 후에는 해제.

UDP(User Datagram Protocol) : 연결 설정 없이 데이터를 바로 전송 가능

Transport Layer에서 중요한 6가지 기능

Process-level의 addressing

multiplexing and demultiplexing 동시에 처리 가능

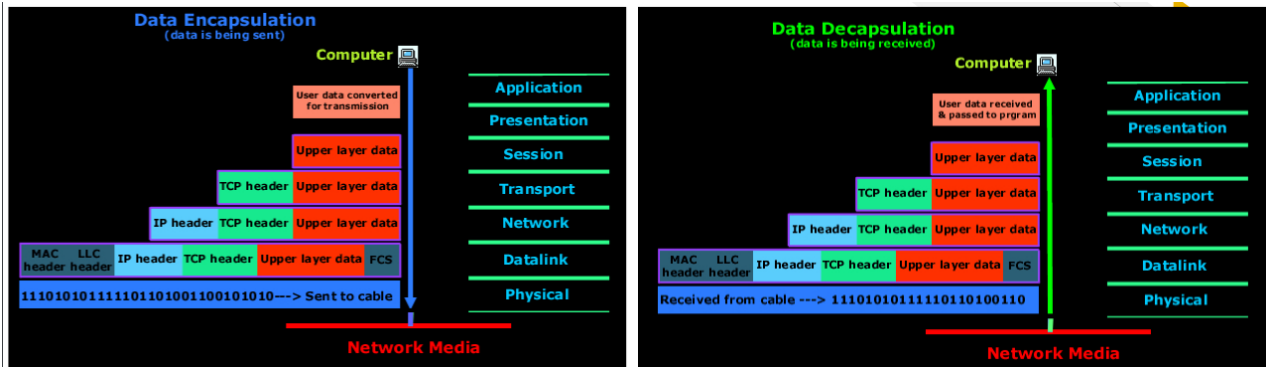
segmentation(쪼개기), packaging(합치기), reassembling(합치기)

연결 설정, 관리, 종료

흐름 관리(병목현상 등)

이더넷(IP, TCP 헤더)

MTU(Maximum transmission unit) : 최대 전송가능 유닛 =1500byte-TCP/IP 헤더~~1460byte
Encapsulation : Layer가 내려오면 내려올수록 헤더가 점점 붙는다.



TCP :

1:40:00~ 다시듣기